

Robot Dynamics & Control

2026

Session 3 – Newton-Euler Recursive Dynamics

Tutors:

Pedro Luis Figueroa Saire

Danial Sabzevari

Emails: name.surname@edu.unige.it (or Teams)

Dibris



UNIVERSITÀ DEGLI STUDI
DI GENOVA

The Newton-Euler Algorithm

The algorithm **visits** the **multibody structure recursively**:

Forward steps (from base to last link)

- Angular velocity
- Angular **acceleration**
- Linear **acceleration**
- Linear **acceleration** of the COM
- Cartesian Force and Torque acting on COM

You want to move the robot, so to accelerate its links and parts

Recursive means that the **contributions** related to a link can be computed by considering only the values related to the previous (forward iteration) or the next (backward iteration) body.

Applying **Newton's II Law (force)** and **Euler Equation (torque)**

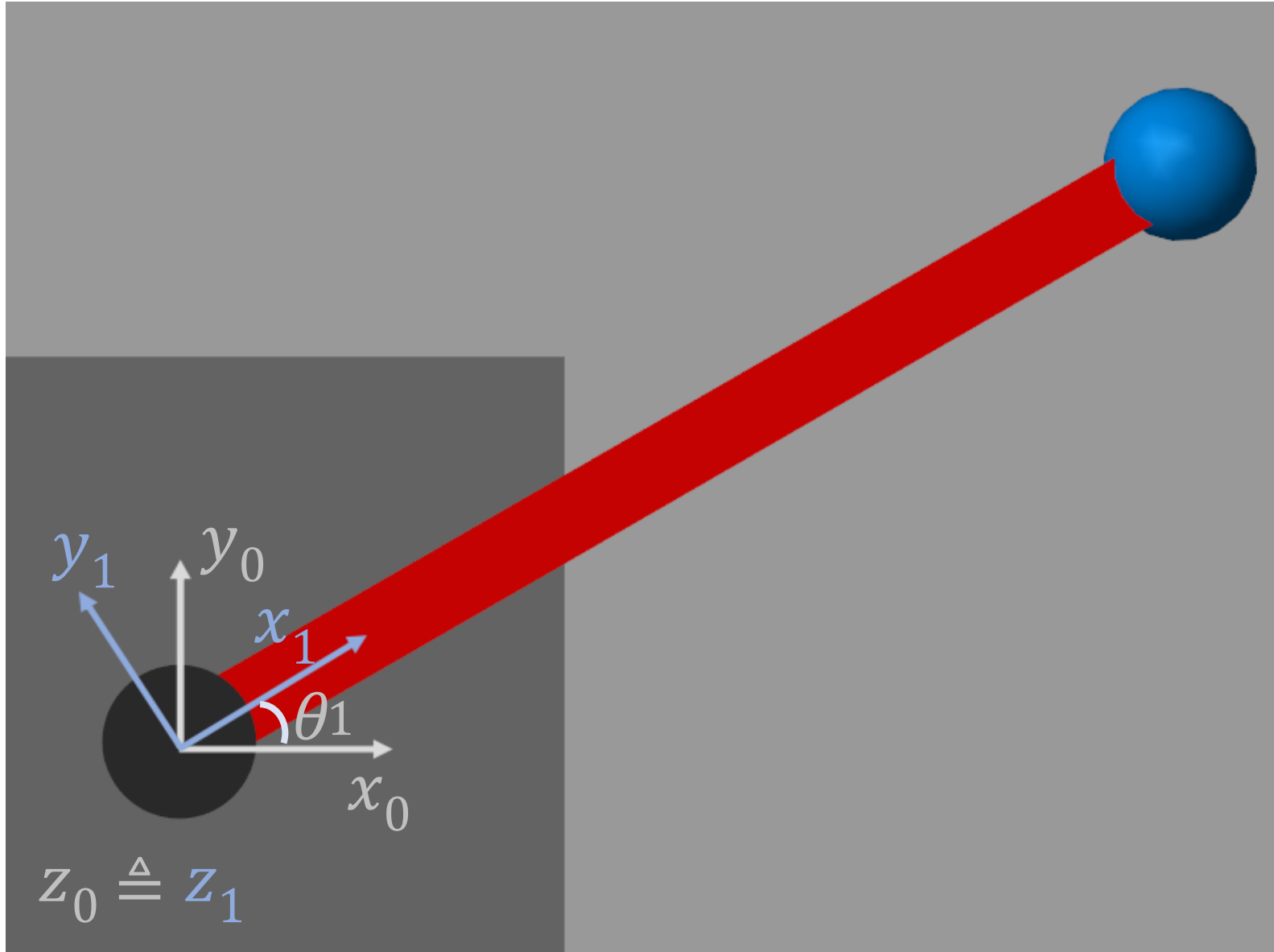
Backward steps (from last link to base)

- **Cartesian Force** and **Torque** acting on **Joints**
- **Generalized Force** acting on the **Joint axis**

This is what you need to apply to actuators to control the robot



Step-by-step Pendulum Dynamics



Convention: Each link's frame is centered on one end, and its position is coincident with the Joint frame position.

The described example shows how to **compute the full dynamic model of the 1R robot in closed-form.**

Exercise: After the tutorial you should implement a recursive algorithm. See hints on slide 16.



Step 0 – Initialization

Necessary quantities for our case of study:

- Positions of Centers of Mass (each one in its link frame):

$${}^1P_{C_1} = \frac{l_1}{2}x_1$$

- Inertia Tensors:

Point mass assumption

$${}^{C_1}I_1 = 0$$

or Uniform mass assumption

$${}^{C_1}I_1 = \begin{bmatrix} i_{1x} & 0 & 0 \\ 0 & i_{1y} & 0 \\ 0 & 0 & i_{1z} \end{bmatrix}$$

Hint: In this case you have a rigid body, so **consider inertia**

- Forces on end-effector:

$$f_2 = 0$$

$$n_2 = 0$$

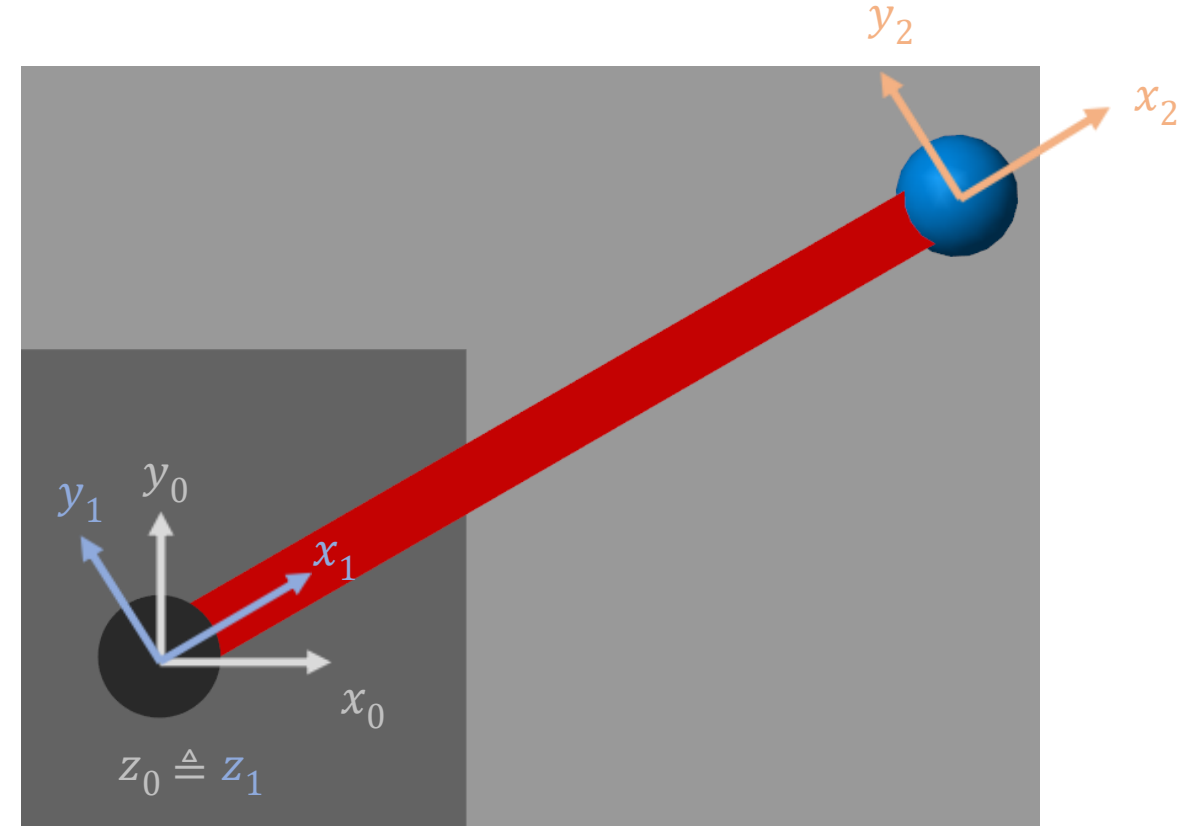
Because **no force** is **acting on the sphere**

- Fixed base robot assumption:

$$\omega_0 = 0$$

$$\dot{\omega}_0 = 0$$

It's the **first link** of the structure



Step 1 – Angular Velocity

FORWARD RECURSIVE STEPS

$${}^{i+1}\omega_{i+1} = \underbrace{{}_i^{i+1}R {}^i\omega_i}_{\text{Previous contribution}} + \underbrace{\dot{\theta}_{i+1} {}^{i+1}Z_{i+1}}_{\text{Current Joint contribution}}$$

In our case we need to compute the angular velocity of Link 1 in Joint 1 frame:

$${}^1\omega_1 = \cancel{{}_0^1R} {}^0\omega_0 + \dot{\theta}_1 {}^1Z_1 \longrightarrow {}^1\omega_1 = \dot{\theta}_1 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 \end{bmatrix}$$

0



Step 2 – Angular Acceleration

$${}^{i+1}\dot{\omega}_{i+1} = \underbrace{{}^i R^{i+1} \dot{\omega}_i + {}^i R^{i+1} \omega_i \times \dot{\theta}_{i+1} {}^{i+1}\hat{Z}_{i+1}}_{\text{Previous joint contribution}} + \underbrace{\ddot{\theta}_{i+1} {}^{i+1}\hat{Z}_{i+1}}_{\text{Current Joint contribution}}$$

In our case we need to compute the angular acceleration of Link 1 in Joint 1 frame:

$${}^1\dot{\omega}_1 = \cancel{{}^1_0 R {}^0\dot{\omega}_0} + \cancel{{}^1_0 R {}^0\omega_0 \wedge \dot{\theta}_1 {}^1\hat{Z}_1} + \ddot{\theta}_1 {}^1\hat{Z}_1 = \begin{bmatrix} 0 \\ 0 \\ \ddot{\theta}_1 \end{bmatrix}$$

$\searrow$
0
 $\searrow$
0



Step 3 – Linear Acceleration

$$\begin{aligned} s1 &\triangleq \sin \theta_1 \\ c1 &\triangleq \cos \theta_1 \end{aligned}$$

$${}^{i+1}\dot{v}_{i+1} = {}^i R^{i+1} ({}^i \dot{\omega}_i \times {}^i P_{i+1} + {}^i \omega_i \times ({}^i \omega_i \times {}^i P_{i+1}) + {}^i \dot{v}_i)$$

In our case we need to compute the linear acceleration of Link 1 in Joint 1 frame:

$${}^1\dot{v}_1 = {}^1_0 R \left[\underset{0}{\cancel{{}^0\dot{w}_0}} \wedge \underset{0}{\cancel{{}^0P_1}} + {}^0w_0 \wedge ({}^0w_0 \wedge {}^0P_1) + {}^0\dot{v}_0 \right] =$$

0 with **NO** gravity

$$= {}^1R_0 {}^0\dot{v}_0 = \begin{bmatrix} c1 & -s1 & 0 \\ s1 & c1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ g \\ 0 \end{bmatrix} = \begin{bmatrix} -g s1 \\ g c1 \\ 0 \end{bmatrix}$$

With gravity, which can be modeled in the algorithm as an acceleration acting on the base frame



Step 4 – Linear Acceleration of COM

$${}^{i+1}\dot{v}_{C_{i+1}} = \underbrace{{}^{i+1}\dot{\omega}_{i+1} \times {}^{i+1}P_{C_{i+1}}}_{\text{Centrifugal term}} + \underbrace{{}^{i+1}\omega_{i+1} \times ({}^{i+1}\omega_{i+1} \times {}^{i+1}P_{C_{i+1}})}_{\text{Centripetal term}} + \underbrace{{}^{i+1}\dot{v}_{i+1}}_{\text{Step 3}}$$

In our case we need to compute the linear acceleration of CoM 1 in Joint 1 frame:

$$\begin{aligned} {}^1\dot{v}_{c_1} &= {}^1\dot{\omega}_1 \wedge {}^1P_{c_1} + {}^1\omega_1 \wedge ({}^1\omega_1 \wedge {}^1P_{c_1}) + {}^1\dot{v}_1 \\ &= \begin{bmatrix} 0 \\ 0 \\ \ddot{\theta}_1 \end{bmatrix} \wedge \begin{bmatrix} \frac{l_1}{2} \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 \end{bmatrix} \wedge \left(\begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 \end{bmatrix} \wedge \begin{bmatrix} \frac{l_1}{2} \\ 0 \\ 0 \end{bmatrix} \right) + \begin{bmatrix} -gs1 \\ gc1 \\ 0 \end{bmatrix} = \\ &= \begin{bmatrix} 0 \\ \frac{l_1}{2}\ddot{\theta}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 \end{bmatrix} \wedge \begin{bmatrix} 0 \\ \frac{l_1}{2}\dot{\theta}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} -gs1 \\ gc1 \\ 0 \end{bmatrix} = \\ &= \begin{bmatrix} 0 \\ \frac{l_1}{2}\ddot{\theta}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} -\frac{l_1}{2}\dot{\theta}_1^2 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -gs1 \\ gc1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{l_1}{2}\dot{\theta}_1^2 - gs1 \\ \frac{l_1}{2}\ddot{\theta}_1 + gc1 \\ 0 \end{bmatrix} \end{aligned}$$

With a **revolute** joint, the attached **link COM will move on a circumference** centered in the joint axis, so it will have both a **centrifugal** and a **centripetal** term



Step 5 – Force acting on COM

$${}^{i+1}F_{i+1} = m_{i+1} {}^{i+1}\dot{v}_{C_{i+1}}$$

Compute the **forces acting at CoM1**

Simply apply the **Newton's II Law**

$${}^1F_1 = m_1 {}^1v_{c_1} = \begin{bmatrix} -m_1 \frac{l_1}{2} \dot{\theta}_1^2 - m_1 g s_1 \\ m_1 \frac{l_1}{2} \ddot{\theta}_1 + m_1 g c_1 \\ 0 \end{bmatrix}$$



Step 6 – Torque acting about COM

$${}^{i+1}N_{i+1} = C_{i+1} I_{i+1} {}^{i+1}\dot{\omega}_{i+1} + \underbrace{{}^{i+1}\omega_{i+1} \times C_{i+1} I_{i+1} {}^{i+1}\omega_{i+1}}$$

Robot is planar, **body is symmetric**, so this is a **cross product between two parallel vectors**, which is **null**

Compute the torques acting on CoM1

Apply **Euler equation**

$${}^1N_1 = {}^{c_1} I_1 {}^1\dot{\omega}_1 = \left\{ \begin{array}{l} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \\ \begin{bmatrix} 0 \\ 0 \\ i_{1z} \ddot{\theta}_1 \end{bmatrix} \end{array} \right.$$

for Point Mass

for a symmetric body about z

END of the FORWARD RECURSIVE ALGORITHM



Step 7 – Force acting on Joint

BACKWARD RECURSIVE ALGORITHM

$${}^i f_i = \underbrace{{}^i_{i+1} R^{i+1} f_{i+1}}_{\text{Next link's contribution}} + \underbrace{{}^i F_i}_{\text{Computed in Step 5}}$$

Compute the force acting on the frame of Joint 1, given those acting on the CoM1:

$${}^1 f_1 = \cancel{{}^1_2 R} {}^2 f_2 + {}^1 F_1 = \begin{bmatrix} -m_1 \frac{l_1}{2} \dot{\theta}_1^2 - m_1 g s_1 \\ m_1 \frac{l_1}{2} \ddot{\theta}_1 + m_1 g c_1 \\ 0 \end{bmatrix}$$

This is **needed** in case of a **Prismatic joint**, in case of a **Revolute joint**, a cartesian force acting on its rotation axis does not produce any motion



Step 8 – Torque acting on Joint

$${}^i n_i = \underbrace{{}^i N_i + {}^i_{i+1} R^{i+1} n_{i+1}}_{\text{Reaction constraint}} + \underbrace{{}^i P_{C_i} \times {}^i F_i}_{\text{Torque from CoM1}} + \underbrace{{}^i P_{i+1} \times {}^i_{i+1} R^{i+1} f_{i+1}}_{\text{Torque due to force on next joint frame}}$$

In our case we need to compute the torque about the frame of Joint 1 , given the forces/torques acting on the CoM1:

$$\begin{aligned} {}^1 n_1 &= {}^1 N_1 + \cancel{{}^1_2 R^2 n_2} + {}^1 P_{c_1} \wedge {}^1 F_1 + \cancel{{}^1 P_2 \wedge {}^1_2 R^2 f_2} = \\ &= {}^1 N_1 + {}^1 P_{c_1} \wedge {}^1 F_1 = \begin{bmatrix} 0 \\ 0 \\ i_{1z} \ddot{\theta}_1 \end{bmatrix} + \begin{bmatrix} \frac{l_1}{2} \\ 0 \\ 0 \end{bmatrix} \wedge \begin{bmatrix} -m_1 \frac{l_1}{2} \dot{\theta}_1^2 - m_1 g s_1 \\ m_1 \frac{l_1}{2} \ddot{\theta}_1 + m_1 g c_1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 0 \\ i_{1z} \ddot{\theta}_1 \end{bmatrix} + \begin{vmatrix} i & j & k \\ \frac{l_1}{2} & 0 & 0 \\ {}^1 F_{1x} & {}^1 F_{1y} & 0 \end{vmatrix} = \begin{bmatrix} 0 \\ 0 \\ i_{1z} \ddot{\theta}_1 + m_1 \frac{l_1}{4} \dot{\theta}_1^2 + m_1 \frac{l_1}{2} g c_1 \end{bmatrix} \end{aligned}$$



Step 9 – Torque acting on Joint motion axis

$$\tau_i = {}^i n_i^T {}^i \hat{Z}_i$$

We have a **Revolute joint**, so it's sufficient to **consider the applied torque** to the **joint frame** and **extract the component about the motion axis** (Z in this case)

In case of a prism-shaped rod of length L and mass M, this is the Inertia seen at the COM, which is $\frac{1}{12}ML^2$

Closed-form Dynamic Model of the 1R robot

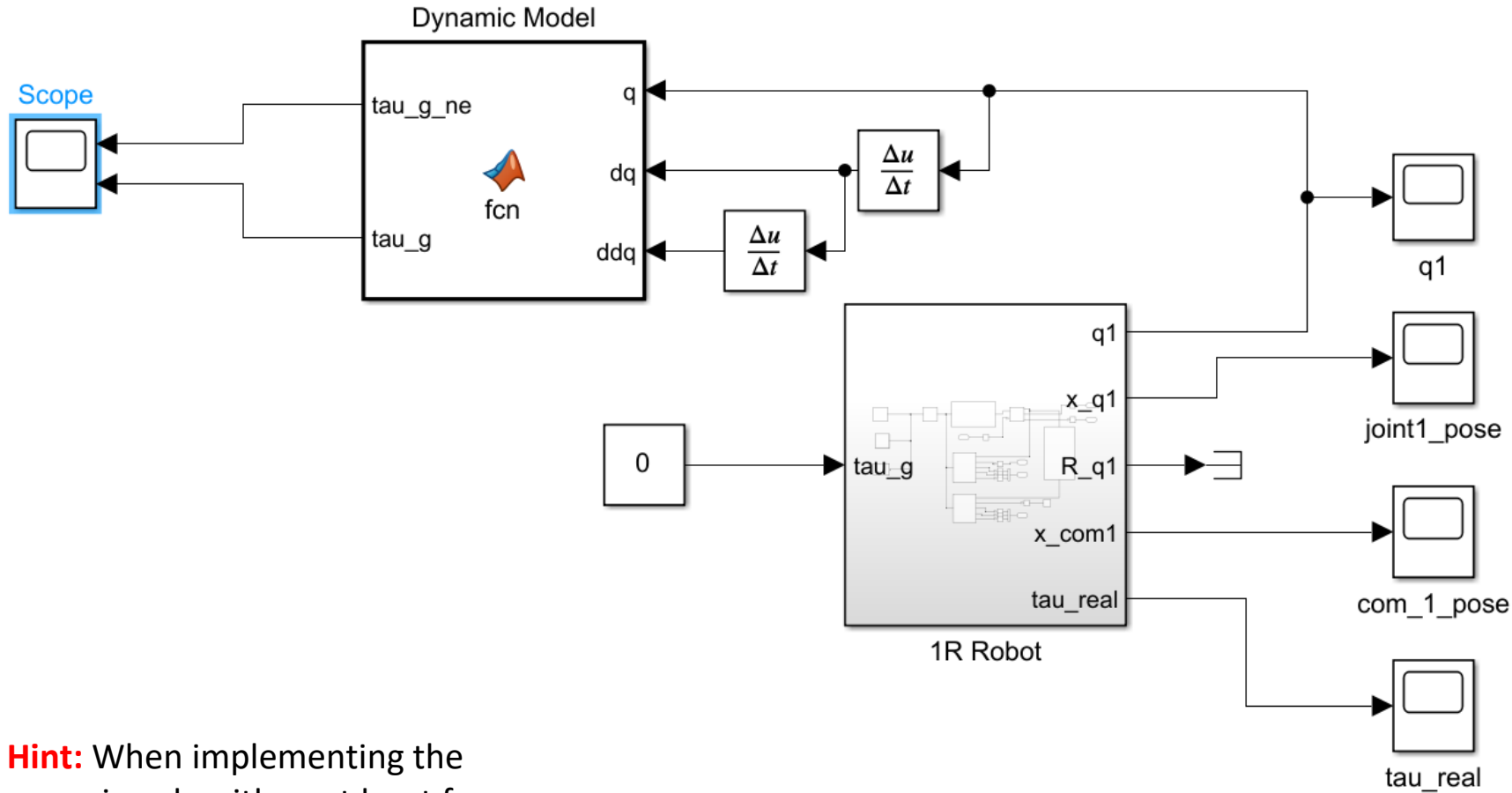
$$\tau_1 = \overset{\uparrow}{i_{1z}} \ddot{\theta}_1 + m_1 \frac{l_1}{4} {}^2 \ddot{\theta}_1 + m_1 \frac{l_1}{2} g c_1$$

This terms are **related** to the **Kinetic Energy** of the system

This term is **related** to the **Potential Energy** of the system



Testing the Closed-Form solution



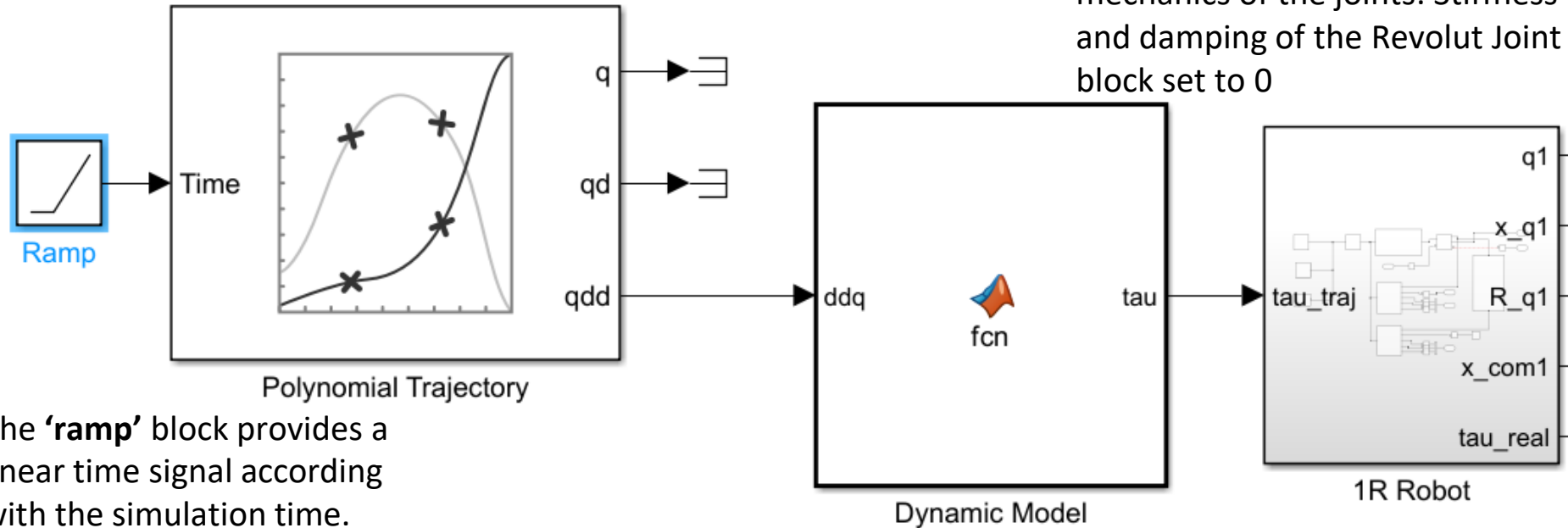
Use the **Derivative Block** on the joint position to obtain its velocity and acceleration

Evaluate its behavior (as an example, here it compares the **gravity compensation torque** obtained with NE with the one obtained with the static law)

Hint: When implementing the recursive algorithm, at least for simple robots, use a closed-form model to double-check the results



Open Loop Trajectory Generation



Hint: Assume ideal internal mechanics of the joints: Stiffness and damping of the Revolut Joint block set to 0

You can **evaluate the correctness of the dynamic model** by applying a **torque** computed to **follow a proper trajectory over time**.

Hint: To have a **modular evaluation** of the motion **disable gravity** in the **mechanism configuration** and assess the trajectory

The '**ramp**' block provides a linear time signal according with the simulation time.

The '**Polynomial Trajectory**' block provides a time-based **smooth trajectory** through waypoints.

This means 'go from 0 to 0.78 rad ($\pi/4$) in 10 seconds' (which is the simulation time)

Waypoints	
Waypoint source:	Internal
Waypoints:	[0, 0.78] [0,0.78]
Time points:	[0, 10] [0,10]
Parameters	
Method:	Cubic Polynomial
Parameter source:	Internal
Velocity boundary conditions:	zeros(1, 2) [0,0]
Simulate using:	Interpreted execution

IMPORTANT: This is **NOT ROBOT CONTROL!** You are just **specifying an open-loop acceleration profile and computing a torque to follow it**, with no guarantees on stability. It's just a test to check that the model is correct.



Exercise – Recursive Implementation

```
function tau = fcn(ddq)

% ----- Robot model quantities and initialization

% ----- Forward iterations
% Loop over the number of links, from 0 to n-1

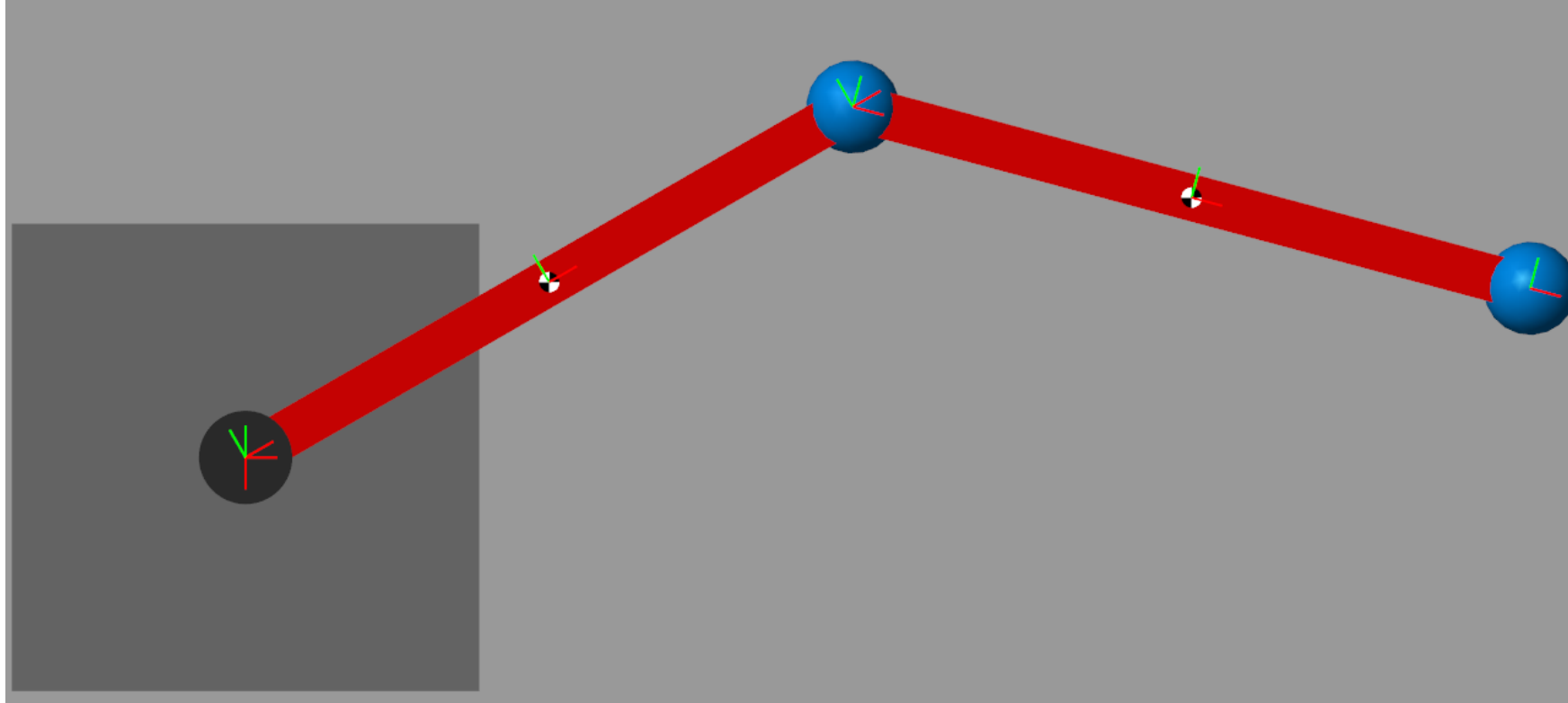
% ----- Backward iterations
% Loop over the number of links, from n to 1
```

Hint: Pre-allocate all the needed data structures before looping over the mechanism.

Hint: This is just a template, you should select properly all the inputs to the block, to get all the needed quantities from the Simscape model.



Exercise – 2R Robot



Again, use the same structure built in the second exercise session

Try to compute a closed-form model to verify your recursive implementation

